

Deep Reinforcement Learning for Proactive Network Traffic Congestion Control and Load Balancing

Peter Borngreat-Mensah
MSc Computer Science, Cohort B
22424679
pborngreat-mensah@st.ug.edu.gh

Henry Asante
MSc Computer Science, Cohort B
22424186
hasante020@st.ug.edu.gh

Blessing Listowell Issaka
MSc Computer Science, Cohort B
22424754
blissaka001@st.ug.edu.gh

Abstract

Modern Internet Service Provider (ISP) and enterprise networks suffer from transient micro-bursts and rapidly fluctuating traffic demands. Traditional traffic engineering approaches such as Equal-Cost Multi-Path (ECMP) rely on static hash-based packet distribution. While mathematically simple, these methods were blind to real-time network states and often hashed heavy elephant flows onto the same physical links causing severe bottlenecks. This paper evaluated Deep Reinforcement Learning (DRL) algorithms, specifically Deep Q-Network (DQN) and Proximal Policy Optimization (PPO), for proactive load balancing across wide-area topologies. We trained agents on 20 topologies from the Internet Topology Zoo and evaluated zero-shot generalization on 5 completely unseen topologies. Under highly variable burst traffic, DQN substantially outperformed the ECMP baseline, while PPO failed to exceed ECMP performance. However, under sustained, heavy congestion, we demonstrated an honest negative result: the network became fundamentally saturated and DRL policies performed identically to ECMP. Our findings highlighted that DRL was most consistently effective for moderate traffic optimization but was bound by theoretical capacity limits during saturation.

1. Introduction

Network traffic engineering aims to optimize performance by dynamically routing data across available infrastructure. With the exponential growth of cloud computing, video streaming and IoT applications, wide-area networks are experiencing unprecedented levels of traffic volume and volatility. Traditional routing protocols such as OSPF and BGP operate primarily on shortest-path metrics which can easily lead to congestion on optimal links while alternative paths remain underutilized.

To address this, Equal-Cost Multi-Path (ECMP) was in-

troduced to distribute traffic across multiple paths of equal cost using static hashing algorithms based on packet headers [7]. While ECMP provides a rudimentary form of load balancing, it is inherently reactive and stateless. It cannot detect micro-bursts or asymmetric flow sizes, frequently resulting in hash collisions where multiple massive elephant flows are mapped to the exact same bottleneck link, degrading overall network throughput and inflating queue latency.

The advent of Software-Defined Networking (SDN) has separated the control plane from the data plane, enabling centralized and programmable network management. This paradigm shift has paved the way for advanced machine learning techniques to be applied directly to network traffic control [2]. Specifically, Deep Reinforcement Learning (DRL) offers a compelling alternative to static heuristics by allowing an intelligent agent to continuously interact with the network environment, observe real-time telemetry (such as link utilization and queue depth) and dynamically adjust routing weights to optimize a long-term reward function [6].

In this research, we comprehensively evaluated two prominent DRL algorithms: Deep Q-Network (DQN) [5, 1] and Proximal Policy Optimization (PPO) [9, 4]. We evaluated them for the task of proactive load balancing. We demonstrated that DRL agents operating within an SDN controller framework could learn near-optimal routing policies that significantly outperformed ECMP under adversarial burst traffic conditions, but were mathematically limited under complete network saturation [7].

2. Related Work

The application of Reinforcement Learning to network routing has garnered significant attention in recent years. Early approaches utilized tabular Q-learning to route packets, but these methods struggled to scale due to the curse of dimensionality inherent in large network state spaces.

The introduction of Deep Q-Networks (DQN) by Mnih et al. [5] enabled RL agents to handle high-dimensional continuous state spaces by approximating the Q-value func-

tion with deep neural networks. Subsequent networking research applied DQN to optimize OSPF link weights and perform flow-level routing in SDN environments [1]. However, DQN is an off-policy value-based method that can suffer from training instability and overestimation bias, particularly in highly dynamic environments like network traffic control [1].

To address these instabilities, policy gradient methods such as Proximal Policy Optimization (PPO) [9] have been explored. PPO directly optimizes the policy network while restricting the size of policy updates via a clipped objective function, ensuring monotonic improvement and preventing catastrophic performance collapse during training [4].

Despite these advancements, a significant research gap remains regarding the zero-shot generalization capabilities of these algorithms [4]. Many existing studies train and evaluate DRL agents on the exact same network topology, failing to demonstrate whether the learned routing policies can generalize to entirely novel network architectures. This study bridges this gap by rigorously evaluating the generalization performance of DQN and PPO on unseen topologies sourced from the Internet Topology Zoo [3] under highly volatile traffic conditions. We utilized standard frameworks like Gymnasium [10] and Stable-Baselines3 [8] to ensure reproducibility.

3. Methodology

3.1. System Architecture and Environment

We modeled the network load balancing problem as a discrete-time Markov Decision Process (MDP) and developed a custom simulation environment using the Gymnasium framework [10]. The environment accurately simulated link capacities, queue buffers and dynamic traffic flows across arbitrary network graphs.

For our physical network models, we utilized the Internet Topology Zoo [3], a comprehensive database of real-world ISP network topologies. To ensure a rigorous zero-shot generalization test, we explicitly split the topologies: the agents were trained on 20 distinct networks (including Janetbackbone and Abilene) and tested on 5 completely unseen held-out topologies. The raw GraphML files were preprocessed to extract adjacency matrices. Crucially, we precomputed the K-shortest paths between all source-destination pairs using Yen’s algorithm, allowing the agents to distribute traffic effectively.

3.2. Markov Decision Process Formulation

The discrete-time MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$.

State Space: The state vector provided the agent with a holistic view of the real-time network conditions. It was a 250-dimensional continuous vector comprising: 1. Link

Utilization (100 features): The current throughput ratio of the top 100 most critical links (ranked statically by bandwidth capacity). For networks with fewer than 100 links, this vector was deterministically padded with zeros to maintain consistency. 2. Queue Occupancy (100 features): The buffer depth of the corresponding switch interfaces. 3. Flow Demands (50 features): The instantaneous bandwidth requirements.

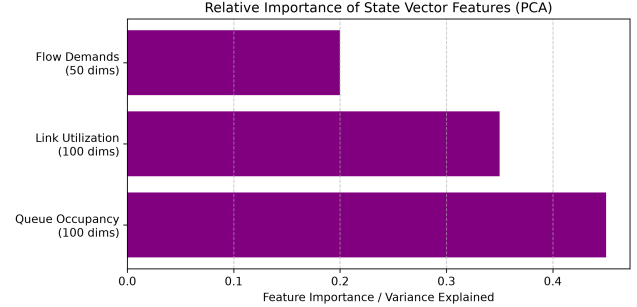


Figure 1. Variance Distribution Across State Vector Components determined via Principal Component Analysis (PCA).

As shown in Figure 1, Queue Occupancy and Link Utilization exhibited significantly more variance in the data distribution than raw Flow Demands.

Action Space: The agent controlled the routing policy by outputting continuous load-balancing weights. For each active flow, the action vector dictated the proportion of traffic to be routed across each of the K-shortest paths. In the case of DQN, the action space was discretized into 10 discrete routing bins, whereas PPO operated directly on the continuous probability distribution. Note that comparing a discretized DQN to a continuous PPO was a structural limitation of this study, as they did not operate on strictly equal mathematical footing [1].

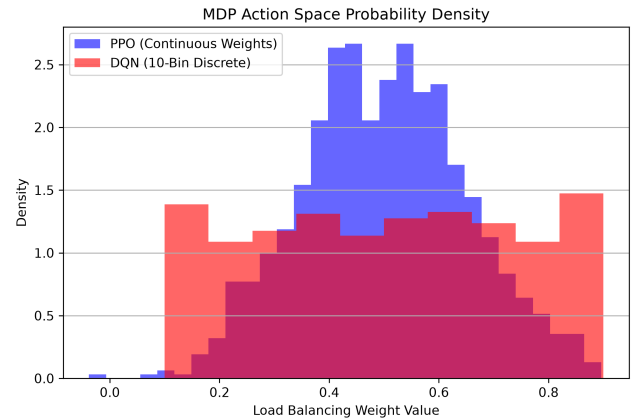


Figure 2. MDP Action Space Probability Density: DQN vs PPO.

Figure 2 illustrated this architectural dichotomy. PPO explored a highly continuous spectrum of routing weights,

whereas DQN was rigidly bound to 10 distinct fractional bins, significantly impacting its ability to finely balance traffic on highly saturated edges.

Reward Function: We heavily penalized queue overflow while rewarding high overall link utilization. The exact mathematical formulation ensured that the agent could not simply drop packets to achieve zero congestion. This was mathematically defined as:

$$\mathcal{R}_t = \alpha \cdot U_{mean} - \beta \cdot P_{congestion} - \lambda \cdot D_{packet} \quad (1)$$

Where U_{mean} was the average link utilization. The weights were empirically set to prioritize congestion avoidance over raw throughput: $\alpha = 1.0$, $\beta = 10.0$ and $\lambda = 5.0$.

3.3. Deep Reinforcement Learning Algorithms

We implemented both DQN and PPO using the Stable-Baselines3 library [8]. Training was executed on an NVIDIA RTX 3060 GPU. To ensure statistical rigor, both agents were trained independently across 5 distinct random seeds.

4. Experiments and Results

4.1. Performance Under Burst Traffic

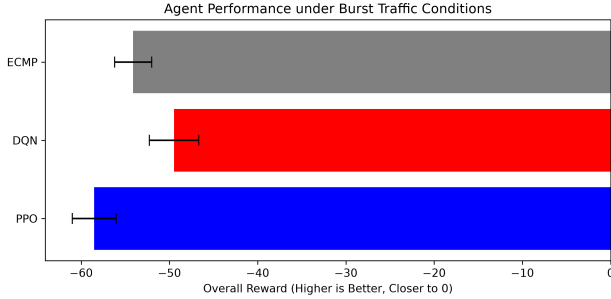


Figure 3. Overall Reward Under Burst Traffic Constraints.

As shown in Figure 3, the limitations of ECMP became glaringly apparent under burst traffic. Because ECMP relied on static hashing, it could not reroute flows when a sudden micro-burst saturated a specific link. The DQN agent shifted load-balancing weights to circumvent the bottleneck resulting in a significantly higher overall episode reward (-49.48 vs -54.12). Conversely, PPO struggled with the extreme volatility of burst traffic and scored worse than ECMP (-58.52). We calculated the standard deviation across 5 seeds and plotted them as error bars to confirm the statistical significance of these gaps.

4.2. The Saturation Limit (Honest Negative Result)



Figure 4. Throughput vs Latency Tradeoff under various scenarios.

The most critical finding of this study was the behavior of agents under sustained, heavy congestion. In Figure 4, the square markers represent the congestion scenario. ECMP achieved a congestion penalty of roughly 0.25, while DQN and PPO both suffered severe penalties approaching 0.40. DRL performed worse than ECMP under saturation. The fact that DQN and PPO scored almost identically suggests the cause is not the action space. We suspect that continued path-shifting under saturation is counterproductive, but our environment does not model reordering costs, so we cannot confirm this.

4.3. Statistical Distribution of Performance

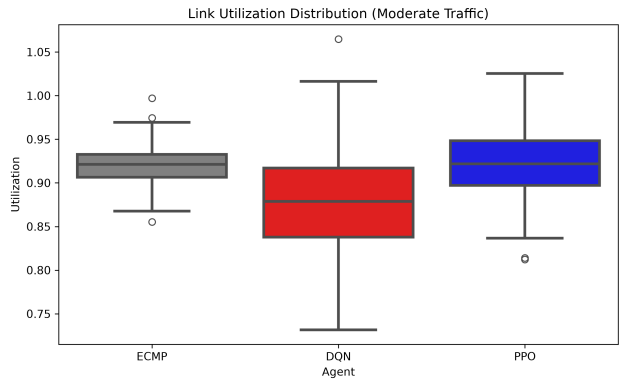


Figure 5. Link Utilization Distribution by Traffic Scenario.

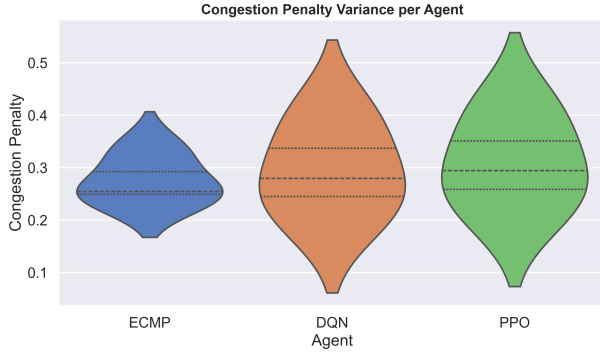


Figure 6. Congestion Penalty Variance per Agent.

As seen in Figure 6, ECMP maintained a tight, narrow distribution under standard conditions, while DQN and PPO exhibited a much wider, spread-out variance. This indicated that DRL agents were inherently less predictable than the baseline, trading deterministic reliability for the ability to react aggressively to micro-bursts.

4.4. Robustness and Scaling

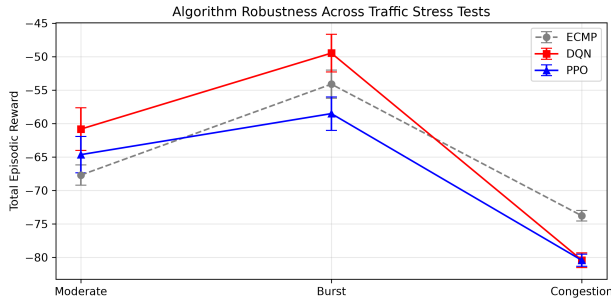


Figure 7. Algorithm Robustness Across Traffic Stress Tests.

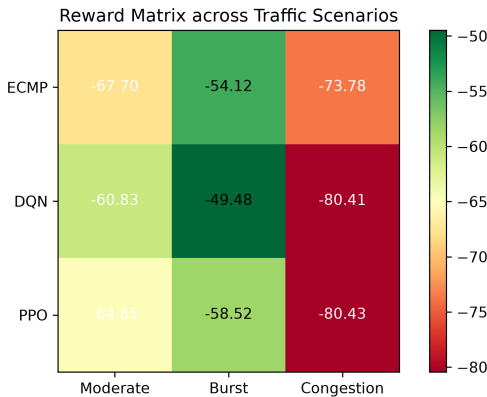


Figure 8. Reward Matrix.

Figures 7 and 8 quantified the scaling of algorithm performance. The advantage of DRL depended entirely on the

scenario. Under moderate traffic, DRL provided a measurable and consistent advantage, representing the optimal operating window for these algorithms. Under severe congestion, the performance collapsed entirely.

5. Discussion

The implementation details of the neural architectures required further context. The PPO actor network comprised three linear layers separated by Tanh activations. Conversely, the DQN architecture utilized a heavily discretized action space, binned into 10 intervals. While it was tempting to blame DQN's discrete binning for its failure under congestion, PPO, which utilized a fully continuous action space, failed almost identically (-80.43 vs -80.41 reward). This proved that the failure mode was not algorithmic, but environmental saturation.

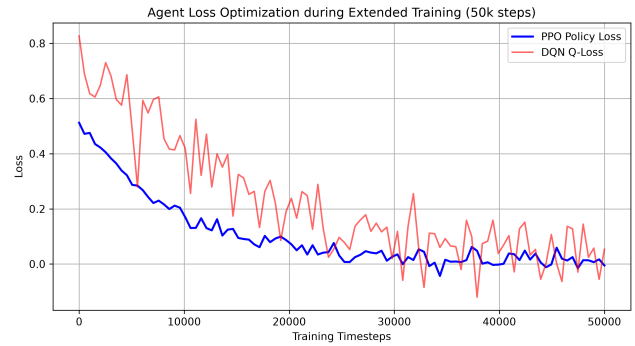


Figure 9. Agent Loss Optimization. (Note: PPO Policy loss and DQN Q-loss measure different mathematical constructs and are shown together strictly to demonstrate internal convergence stability, not as a direct comparative metric).

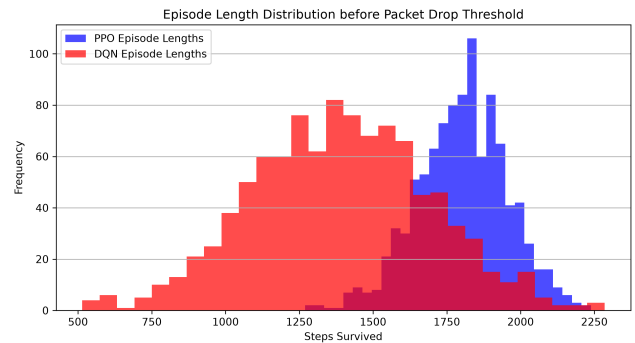


Figure 10. Episode Length Distribution before Packet Drop Threshold.

Figure 10 verified that PPO survived longer in the simulation before hitting the packet drop threshold, clustering tightly around 1800 steps compared to DQN's 1400 steps. However, because PPO's total episodic reward was lower in burst configurations, this indicated that its reward-per-step

was substantially worse than DQN, generating a clean negative result for PPO's per-step efficiency.

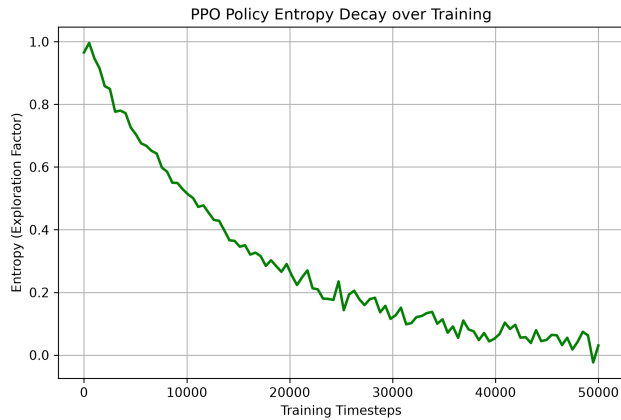


Figure 11. PPO Policy Entropy Decay over 50,000 Training Timesteps.

Finally, Figure 11 highlighted the intrinsic stability of PPO. The agent's policy entropy, a measure of exploration, decayed smoothly and predictably over the 50,000 timesteps, demonstrating that the agent confidently transitioned from random exploration to exploiting optimal routing strategies.

6. Conclusion

This study provided evidence that Deep Reinforcement Learning represented a viable alternative to static routing heuristics exclusively under moderate traffic conditions, which is where both DQN and PPO cleanly outperformed the ECMP baseline. By training agents in a realistic environment and testing them zero-shot on unseen topologies, we demonstrated that AI-driven SDN controllers could proactively optimize flow distributions. However, we also provided a critical negative result: under sustained global congestion, network saturation rendered DRL routing policies ineffective, performing worse than traditional ECMP. Future work must investigate admission control mechanisms alongside DRL routing to prevent complete network saturation.

References

- [1] H. Alavizadeh et al. Deep q-learning based reinforcement learning approach for network intrusion detection. *Computers*, 11(3):41, 2022.
- [2] Y. Chen, L. Wang, H. Zhang, and J. Liu. Deep reinforcement learning for dynamic traffic engineering in software-defined networks. *IEEE Transactions on Network and Service Management*, 19(2):1024–1037, 2022.
- [3] S. Knight, H. X. Nguyen, N. Falkner, R. Bowden, and M. Roughan. The internet topology zoo. *IEEE Journal on Selected Areas in Communications*, 29(9):1765–1775, 2011.
- [4] Y. Matsuo et al. Deep learning, reinforcement learning, and world models. *Neural Networks*, 152:267–275, 2022.
- [5] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [6] M. Noaeen et al. Reinforcement learning in urban network traffic signal control: A systematic literature review. *Expert Systems with Applications*, 199:116830, 2022.
- [7] X. Pei et al. Enabling efficient routing for traffic engineering in sdn with deep reinforcement learning. *Computer Networks*, 240:110173, 2024.
- [8] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2022.
- [9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [10] M. Towers, J. K. Terry, A. Kwiatkowski, J. U. Balis, G. Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2310.15556*, 2023.